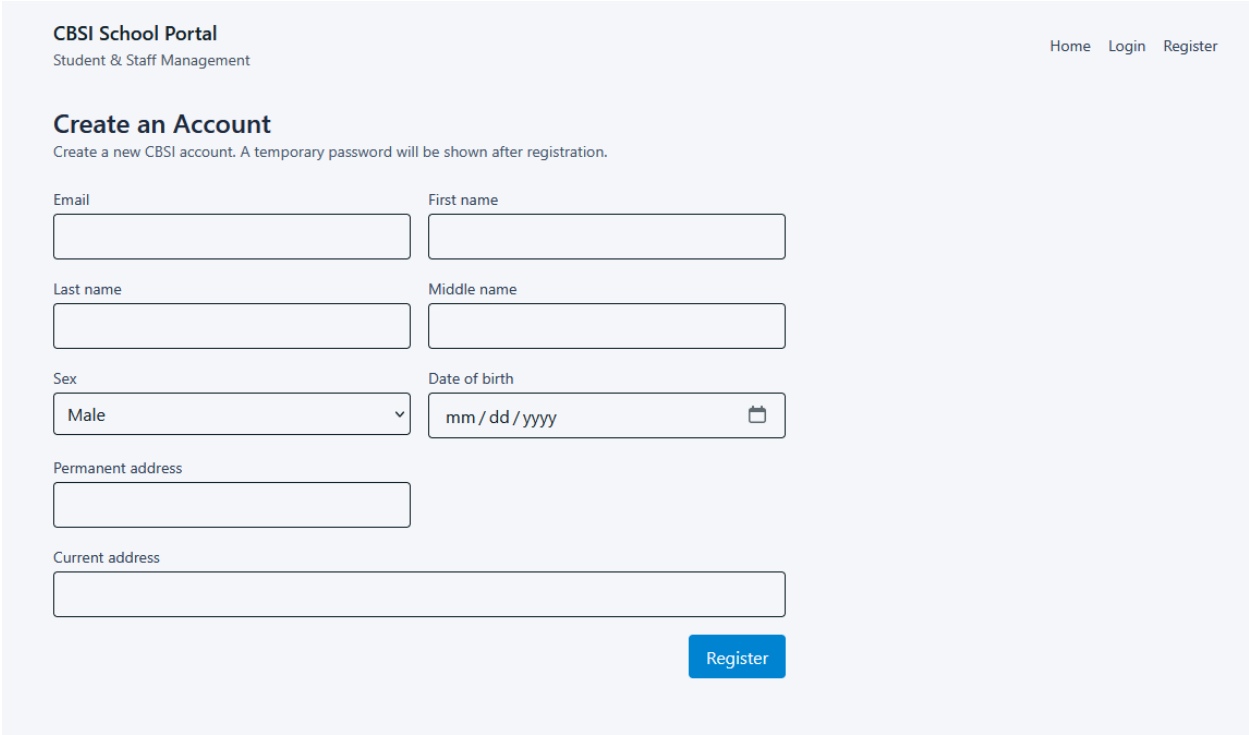
https://github.com/tallbreadstick/IT342-Ramos-CbsiSchoolManagementSystem

```
SUPABASE_PASSWORD=zmcCNIlrlA10VwVk
JWT_SECRET=BA6do76icSHBhEcEh4Q4f6tKiBrNLnPLqueUXpvUw+c=
```

**Last commit hash: 9982655396a50071c87bdf1d694d1421f5b14653**

**User Registration**

All accounts should be created from the admin-facing side. I will later hide this interface behind a more secure channel, but for now this shows the required info. The admin must provide user info into the form, and the system should generate a school ID for the user (staff or student with their own conventions) and should send them an email containing their initial login credentials. They must later change their password on their first time logging in. Later on, I plan to add a spreadsheet input method so that admins can register accounts to users by batch.




**User Login**

Users can only login to accounts registered by an admin or staff with the right permissions. They must change their password on their first log in to update the *password_changed* field in the database, which must be verified before granting access to any personal data or app functionality. User validation on the client side is verified through json web tokens.

```sql
CREATE TABLE users (
    -- database-generated id
    id SERIAL PRIMARY KEY,

    -- primary human-readable identifiers
    school_id VARCHAR(20) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,

    -- personal information
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    middle_name VARCHAR(255),
    sex sex_type NOT NULL,
    date_of_birth DATE NOT NULL,
    permanent_address TEXT,
    current_address TEXT,

    -- security and metadata
    password_hash VARCHAR(255) NOT NULL,
    password_changed BOOLEAN NOT NULL DEFAULT FALSE,
    date_created TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    date_updated TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    -- account status
    account_status account_status_type NOT NULL DEFAULT 'PENDING',
    last_login TIMESTAMPTZ
);

CREATE OR REPLACE FUNCTION update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
   NEW.date_updated = CURRENT_TIMESTAMP;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER set_timestamp
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_timestamp();
```

This is our database table for users, which should include a system-generated school ID, a serial database ID, personal info such as names, sex, address, DoB, etc. For security, passwords will be hashed in argon2,

users must change their password after first login. Admins and staff should be able to update the account status of other users.